CHAPTER 1

DEEP REINFORCEMENT LEARNING

1 Neural Networks

...

2 Distributional Reinforcement Learning

Remember that we defined $\mathbb{P}^\pi_{s,a} := \mathbb{P}^\pi \otimes \delta_{S_0}(s) \otimes \delta_{A_0}(a)$ as the probability measure of the Markov reward process (S, A, R) started in (s, a). We define the random variable of the return under policy π as

$$Z^\pi := \sum_{t=0}^{\infty} \gamma^t R_t, \quad \gamma \in (0,1).$$

Unlike the methods before, where we were interested in the expected reward $Q^\pi(s,a) = \mathbb{E}^\pi_{s,a}[Z^\pi]$, we are now interested in the distribution of these cumulative rewards. For that define

$$\eta^\pi_{s,a}(B) := \mathbb{P}^\pi_{s,a}(Z^\pi \in B), \quad B \in \mathcal{B}(\mathbb{R}).$$

In analogy to weak solutions for PDEs, i.e. in sense of distribution, the return distribution $\eta^\pi_{s,a}$ satisfies the Bellman equation in sense of distribution:

$$\forall \phi \in C_b(\mathbb{R}): \quad \int_{\mathbb{R}} \phi(z) d\eta^\pi_{s,a}(z) = \mathbb{E}^\pi_{s,a}[\int_{\mathbb{R}} \phi(R + \gamma z) d\eta_{S',A'}(z)].$$

We define $f_{r,\gamma}(z) := r + \gamma z$ and the push forward

$$((\eta^\pi_{s,a})_{f_{r,\gamma}})(B) := \eta^\pi_{s,a}(f^{-1}_{r,\gamma}(B)), \quad B \in \mathcal{B}(\mathbb{R}).$$

then, the above can be written as

$$\begin{aligned}
&\forall \phi \in C_b(\mathbb{R}): \quad \int_{\mathbb{R}} \phi(z) d\eta^\pi_{s,a}(z) = \mathbb{E}^\pi_{s,a}[\int_{\mathbb{R}} \phi(R + \gamma z) d\eta_{S',A'}(z)] = \mathbb{E}^\pi_{s,a}[\int_{\mathbb{R}} \phi(z) d(\eta_{S',A'})_{f_{R,\gamma}}(z)] \\
&\iff \forall \phi \in C_b(\mathbb{R}): \quad \langle \phi, \eta^\pi_{s,a} \rangle = \mathbb{E}^\pi_{s,a}[\langle \phi, (\eta^\pi_{S',A'})_{f_{R,\gamma}} \rangle].
\end{aligned}$$

For any $\phi \in C_b(\mathbb{R})$ and any measure ν we have

$$\begin{aligned}
(\nu * \delta_r)(\phi) &= \int_{\mathbb{R}} \phi(z) d(\nu * \delta_r)(z) = \int_{\mathbb{R}} \int_{\mathbb{R}} \phi(x + y) d\nu(x) d\delta_r(y) \\
&= \int_{\mathbb{R}} \phi(x + r) d\nu(x) = (\nu(\cdot - r))(\phi).
\end{aligned}$$

Next, define $D_\gamma(x) := \gamma x$, then if $Z \sim \eta^\pi(x,a)$ then $\gamma Z \sim (\eta^\pi(s,a))_{D_\gamma}$. In total

$$Z\gamma + r \sim (\eta^\pi(s,a))_{D_\gamma}(\cdot - r) = ((\eta^\pi(s,a))_{D_\gamma} * \delta_r)$$

holds in sense of distribution. Now define the distributional Bellman operator in sense of distribution as

$$\mathcal{T}^\pi Z^\pi(s,a) = R(s,a) + \gamma Z^\pi(S', A'),$$

i.e. the distribution

$$\mathcal{T}^\pi : (\mathcal{P}(\mathbb{R}))^{\mathcal{S}\times\mathcal{A}} \to (\mathcal{P}(\mathbb{R}))^{\mathcal{S}\times\mathcal{A}}, (\nu_{s,a})_{(s,a)\in\mathcal{S}\times\mathcal{A}} \mapsto ((\mathcal{T}^\pi\nu)_{s,a})_{(s,a)\in\mathcal{S}\times\mathcal{A}}$$

is point wise defined for all $\phi \in C_b(\mathbb{R})$ as

$$(\mathcal{T}^\pi\nu)_{s,a}(\phi) := \mathbb{E}^\pi_{s,a}[((\nu(S',A'))_{D_\gamma} * \delta_R)(\phi)] = \sum_{s',a',r} \pi(a';s')p(s',r;s,a)((\nu_{s',a'})_{D_\gamma} * \delta_r)(\phi)$$

3 Deep Q-Networks

Deep Q-Networks (DQN) is a Q-learning variant that utilizes deep neural networks to approximate the Q-value. This algorithm addresses how to deal with large, continuous state action spaces $(\mathcal{S}, \mathcal{A})$. The idea is to reduce dimensionality by plugging in the state space $\mathcal{S}$ (e.g. pixels in an Atari game) into a convolutional neural layer, which is the fed into a feed forward neural network that outputs the Q-value for each action $a \in \mathcal{A}$. Problems with simple neural networks are

- They can forget
- Are unstable: small changes in Q-values can lead to big changes in the policy, i.e. the action distribution
- Correlation: In a trajectory τ_i of a rollout, the consecutive states s_t, s_{t+1} are highly correlated, which violates the i.i.d. assumption for estimating the expectation.
- The loss is non-stationary, since the target values depend on the parameters of the network itself, meaning
$$\mathcal{L}(\theta) := \mathbb{E}[(\mathbb{E}[R + \gamma \max_{a'} Q^\theta(S', a')] - Q^\theta(S,A))^2],$$
where both Q terms depend on θ.

In DQN these issues where solved by the following

- Experience Replay: A behavoiral policy π^b samples Rollouts, that are stored in a buffer $\mathcal{D} = \{(s_i, a_i, r_i, s_i')\}_{i=1}^N$. In order to break the correlation between consecutive states, mini batches are sampled uniformly from this buffer to update the Q-network.
- Target Network: A second network with parameters θ^- is introduced, which is bootstrapped, i.e. every C steps the parameters of the Q-network are copied to the target network $\theta^- \leftarrow \theta$. This delay stabilizes the training.

Tabular Q-learning can be interpreted in the following two ways:

- ML view: Fit a target Q_n to the data Q^{π^*}. Define $Y_n := R + \gamma \max_{a'\in\mathcal{A}} Q_n(S', a')$ and
$$\tilde{L}^Q_n := \mathbb{E}[(\mathbb{E}_{S,A}[Y_n] - Q_n(S,A))^2] = \mathbb{E}[(Y_n - Q_n(S,A))^2] + \mathbb{E}[\mathbb{V}_{S,A}(Y_n)] =: L^Q_n + \mathbb{E}[\mathbb{V}_{S,A}(Y_n)].$$

- RL view: We want to approximate the Bellman optimality operator

$$T^*Q(s,a) := \mathbb{E}_{s,a}[R + \gamma \max_{a' \in \mathcal{A}} Q(S', a')].$$

  Because it is a contraction in $\|\cdot\|_\infty$, the fixed point iteration converges.

Because the expectation is the minimizer of the L^2 error (variance), it holds that $\mathbb{E}[X] = \arg\min_\theta \mathbb{E}[(X-\theta)^2]$ which leads to

$$T^*Q(s,a) = \mathbb{E}_{s,a}[R + \gamma \max_{a' \in \mathcal{A}} Q(S', a')] = \arg\min_\theta \mathbb{E}_{s,a}[(R + \gamma \max_{a' \in \mathcal{A}} Q(S', a') - \theta)^2].$$

Thus doing one step SGD (with step size $\frac{\alpha}{2}$???) on L_n^Q or doing the iteration for Q-learning yields the same scheme:

$$Q_{n+1}(s,a) = Q_n(s,a) + \alpha(y - Q_n(s,a)).$$

Remark 3.1

When doing Q-learning with neural networks, we no longer do the gradient on $(s,a) \in \mathcal{S} \times \mathcal{A}$, but instead on a smaller parameter space $\Theta \subset \mathcal{S} \times \mathcal{A}$. Additionally, parameterizing the target with a different set of parameters θ^- the gradient of the loss becomes

$$\nabla_\theta L_n^Q = \mathbb{E}[\nabla_\theta (Y_n^{\theta^-} - Q_n^\theta(S,A))^2] = -\mathbb{E}[2(Y_n^{\theta^-} - Q_n^\theta(S,A))\nabla_\theta Q_n^\theta(S,A)].???-$$

So the gradient step for SGD becomes

$$Q_{n+1}^\theta(s,a) = Q_n^\theta(s,a) + \alpha(y_n^{\theta^-} - Q_n^\theta(s,a))\nabla_\theta Q_n^\theta(s,a).$$

Remark 3.2

In a convolutional neural network a convolutional layer is given by

$$(f_k * h)(i) = \sum_{j=-n}^{n} f_k(i-j)h(i),$$

where f_i is a filter $k = 1, ..., K$ and h is some input vector. These K filters operator simultaninously to and produce $(f_k * h)(i), k = 1, ..., K$ as output. K is the number of Filters. In this setting the step size is called stride and was in the above 1. In general it is

$$(f_k * h)(i) = \sum_{j=-n}^{n} f_k(i+s-j)h(i), \quad s \in \mathbb{N}.$$

Remark 3.3

For Atari games the Architecture is as follows:

- Preprocessing: The environment 84×84 pixels is converted into grayscale. Let $x_t \in \mathbb{R}^{H \times W}$ be a greyscale image at time t, where $H = W = 84$. To create a sense of motion four consecutive images are stacked: $s_t := (x_t, ..., x_{t-3}) \in \mathbb{R}^{H \times W \times 4}$. Together with the scores, this is fed into the neural network.

- Convolutional Layers: There are three convolutional layers used. Each convolutional layers is followed by a ReLU activation function. The first layer has 32 filters of size 8×8 with stride 4. The second layer has 64 filters of size 4×4 with stride 2. The third layer has 64 filters of size 3×3 with stride 1.
- Feed Forward Layers: The output of the last convolutional layer is flattened and fed into a fully connected layer with 512 units, followed by a ReLU activation function.
- Output: The final layer in the network is a fully connected layer that outputs a vector of Q-values, one for each possible action in the Atari game.

Remark 3.4

- Experience Replay: This solves two problems: From the buffer $\mathcal{D}$ mini batches are chosen following an ϵ-greedy policy, which solves the problem of the correlation between rollouts and the policy. Further, it also solves the problem, that neural networks are prone to forget old data.
  If the buffer is full, the oldest transitions are removed first.
- Target Networks:
  - Different Parametrization: The target distribution is parametrized with θ^- Every N steps it is updated with a soft update rule

    $$\theta^- \leftarrow \tau\theta^- + (1-\tau)\theta.$$

    This stabilizes training:

    $$\tilde{L}_n(\theta) := \mathbb{E}[\Big(\mathbb{E}_{S,A}[Y_n^{\theta^-}] - Q_n^\theta(s,a)\Big)^2] = L_n(\theta) + \mathbb{E}[\mathbb{V}_{S,A}[Y_n^{\theta^-}]],$$

    because $\tilde{L}_n(\theta)$ no longer depends on the expected variance.
  - Update every N steps: With the second point of view of Q learning, updating every N steps is actually N step SGD of

    $$T^*Q^{\theta^-}(s,a) = \mathbb{E}_{s,a}[R + \gamma Q^{\theta^-}(S',a')] = \arg\min_\theta \mathbb{E}[(Y_n^{\theta^-} - \theta)^2].$$

Algorithm 1: The Deep Q-Network (DQN) algorithm.

Data: Replay buffer capacity $|D|$, discount factor γ, exploration rate ε, target update frequency N
Result: Trained Q-network approximating Q^*
Initialise replay buffer D
Initialise Q-network with random weights θ
Copy weights to target network: $\theta^- \leftarrow \theta$
for *each episode* **do**
  Pre-process the initial observation s_1 to obtain state representation ϕ_1
  for *each time-step* t **do**
    Select action a_t using ε-greedy policy based on $Q(\phi_t, a; \theta)$
    Execute action a_t, observe reward r_t and next observation s_{t+1}
    Pre-process s_{t+1} to obtain ϕ_{t+1}
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in replay buffer D
    Sample mini-batch B of transitions from D
    for *each transition* $i \in B$ **do**
      Compute target:
      $$Y_i = \begin{cases} r_i, & \text{if terminal transition,} \\ r_i + \gamma \max_{a'} Q(\phi_i', a'; \theta^-), & \text{otherwise.} \end{cases}$$
    end
    Perform gradient descent on loss:
    $$L_B(\theta) = \frac{1}{|B|} \sum_{i \in B} (Y_i - Q(\phi_i, a_i; \theta))^2$$
    Update Q-network weights θ
    if *every* N *steps* **then**
      Update target network: $\theta^- \leftarrow \theta$
    end
    However, target networks increase memory, as it another neural network needs to be stored and updating the weights increases computational costs.
  end
end

Remark 3.5

- Double Q-learning and Double DQN: Q-learning over estimates the action values due to the max. operator, In Double Q-learning we maintain two independent networks Q^A and Q^B to estimate Q^{π^*}:

$$\begin{aligned} Q^A(s,a) &\leftarrow (1-\alpha)Q^A(s,a) + \alpha\left(r + \gamma Q^B(s', \arg\max_{a' \in \mathcal{A}} Q^A(s',a'))\right) \\ &= (1-\alpha)Q^A(s,a) + \alpha\left(r + \gamma Q^A(s', \arg\max_{a' \in \mathcal{A}} Q^A(s',a'))\right) \\ &+ \gamma\left(Q^B(s', \arg\max_{a' \in \mathcal{A}}(s',a')) - Q^A(s', \arg\max_{a' \in \mathcal{A}} Q^A(s',a'))\right). \end{aligned}$$

In Double Q-learning the Networks are updated in every step and independendly. In Double DQN we give one network the role of the target network and the other the Q-network. The Q-network is updated every step. The target network is updated only every N step with the soft update rule, which depends on the Q-network. So indepence and simultaninous updates do not happen.

- Clipped Double Q/TD3: Another approach is to clip the bias term (third summand in the above) by taking its negative part: $b^- := \min\{b, 0\}$. The deep variant of this method is called Twin delayed double Deep Q Networks.

4 Proximal Policy Gradient

Consider a finite time MDP and the respective Value/Q-functions:

$$V^\pi(s) := \mathbb{E}^\pi_s[\sum_{t=0}^{T-1} \gamma^t R_t], \quad V^\pi_t[\sum_{t'=t}^{T-1} \gamma^{t'} R_{t'}], \quad Q^\pi_t(s,a) = \mathbb{E}^\pi_{s,a}[\sum_{t'=t}^{T_1} \gamma^{t'} R_{t'}],$$

where $(\pi_\theta)^{\theta\in\Theta}$ is a family of differentiable policies. The gradient is then given by

$$\nabla_\theta V^{\pi_\theta}(s) = \sum_{t=0}^{T-1} \gamma^t \mathbb{E}^{\pi_\theta}[\nabla_\theta \log(\pi_\theta(A_t; S_t)) R^T_t], \quad R^T_t := \sum_{i=t}^{T-1} \gamma^{i-t} R_i.$$

The idea is now threefold. Because the gradient is an expectation, we need to sample from it. We want to reuse as many samples as possible, do as few updates of the parameter θ as possible to reduce computational time and reduce the variance of the expectation:

- Reusing samples: We think of PPO as a rollout based stochastic gradient approximation. We reuse old parameters for gradient steps (surragate gradient steps) and show that these biased gradient steps have an advantage. Let D^{π_θ} be the set of Rollouts created by π_θ. Randomly reshuffle this set and decompose it into mini-batches. We pass through this dataset, by doing a gradient step for each mini batch. After we have passed through the entire dataset (one epoch) we set $\theta := \theta_{old}$ and start a new cycle.

- Reduce updates:

- Reduce variance: Subtracting a Basline can reduce variance. We choose the Baseline as the Value function, i.e.

$$\nabla_\theta V^{\pi_\theta}(s) = \sum_{t=0}^{T-1} \gamma^t \mathbb{E}^{\pi_\theta}[\nabla_\theta \log(\pi_\theta(A_t; S_t)) \underbrace{(Q^{\pi_\theta}_t - V^{\pi_\theta}_t)}_{=:A^{\pi_\theta}_t}],$$

  where $A^{\pi_\theta}_t$ is called advantage function.
  Another way is to decouple sampling and updating, using importance sampling:

$$\nabla_\theta V^{\pi_\theta}(s) = \sum_{t=0}^{T-1} \gamma^t \mathbb{E}^{\pi_{\theta^{old}}} \left[\prod_{i=0}^{t} \frac{\pi_\theta(A_i; S_i)}{\pi_{\theta^{old}}(A_i; S_i)} \nabla_\theta \log(\pi_\theta(A_t; S_t)) A^{\pi_\theta}_t(S_t, A_t) \right].$$

Thus the current policy needs to be evaluted and $A^{\pi_\theta}_t$ needs to be computed. The alternation between gradient steps and policy evalutaiton is called actor critic. A2C does this by using neural networks to parameterize

the Advantage function and uses GAE for advantage estimation.
GAE is used for infinite time MDPs. Also used for finite time by truncating at T. Usually γ is ignored, but this is required???

Remark 4.1

Assumptions:

- Bounded rewards: $|R_t| \leq R_*$
- Bounded score function: $\forall \theta \in \mathbb{R}^d, s \in \mathcal{S}, a \in \mathcal{A}: \quad |\nabla_\theta \log(\pi_\theta(a;s))| \leq \Pi_*$
- Lipschitz score function:

$$\forall \theta \in \mathbb{R}^d, s \in \mathcal{S}, a \in \mathcal{A}: \quad |\nabla_\theta \log(\pi_\theta(a;s)) - \nabla_{\theta'} \log(\pi_{\theta'}(a;s))| \leq L_s|\theta - \theta'|.$$

- Access to $\hat{A}_t$ which is bounded by A_*.

$$\begin{aligned}
\nabla_\theta J(\theta) &= \sum_{t=0}^{T-1} \gamma^t \prod_{i=0}^{t} \mathbb{E}^{\pi_{\theta^{old}}}[\frac{\pi_\theta(A_i;S_i)}{\pi_{\theta^{old}}(A_i;S_i)} \nabla_\theta \log(\pi_\theta(A_i;S_i)) A_t^{\pi_\theta}(S_t,A_t)] \\
&\overset{(*)}{\approx} \sum_{t=0}^{T-1} \gamma^t \mathbb{E}^{\pi_{\theta^{old}}}[\frac{\pi_\theta(A_t;S_t)}{\pi_{\theta^{old}}(A_t;S_t)} \nabla_\theta \log(\pi_\theta(A_t;S_t)) A_t^{\pi_{\theta^{old}}}(S_t,A_t)] \\
&\overset{logTrick}{=} \sum_{t=0}^{T-1} \gamma^t \mathbb{E}^{\pi_{\theta^{old}}}[\frac{\nabla_\theta \pi_\theta(A_t;S_t)}{\pi_{\theta^{old}}(A_t;S_t)} A_t^{\pi_{\theta^{old}}}(S_t,A_t)] \\
&\overset{(**)}{\approx} \sum_{t=0}^{T-1} \gamma^t \mathbb{E}^{\pi_{\theta^{old}}}[\frac{\nabla_\theta \pi_\theta(A_t;S_t)}{\pi_{\theta^{old}}} \mathbf{1}_{\left|\frac{\pi_\theta(A_t;S_t)}{\pi_{\theta^{old}}(A_t;S_t)}-1\right| \leq \frac{1}{\epsilon}} A_t^{\pi_{\theta^{old}}}(S_t,A_t)] \\
&= T\mathbb{E}^{U\sim\mathcal{U}(\{0,...,T-1\})}\left[\gamma^U \mathbb{E}^{\pi_{\theta^{old}}}[\frac{\nabla_\theta \pi_\theta(A_U;S_U)}{\pi_{\theta^{old}}} \mathbf{1}_{\left|\frac{\pi_\theta(A_t;S_t)}{\pi_{\theta^{old}}(A_t;S_t)}-1\right| \leq \frac{1}{\epsilon}} A_U^{\pi_{\theta^{old}}}(S_U,A_U)]\right] \\
&=: g_{PPO}^{clip}(\theta,\theta^{old}).
\end{aligned}$$

In the last equality we interpreted $\frac{1}{T}\sum_{t=0}^{T-1}$ as an expactation of a uniform random variable on $\{0,...,T-1\}$. The unclipped version we define as

$$g_{PPO}(\theta,\theta^{old}) := T\mathbb{E}^{U\sim\mathcal{U}(\{0,...,T-1\})}\left[\gamma^U \mathbb{E}^{\pi_{\theta^{old}}}[\frac{\nabla_\theta \pi_\theta(A_U;S_U)}{\pi_{\theta^{old}}} A_U^{\pi_{\theta^{old}}}(S_U,A_U)]\right].$$

Remark 4.2

(*)

Define

$$M_{ean}\, TV(\pi_{\theta^{old}}, \pi_\theta) := \frac{1}{T}\sum_{t=0}^{T-1} \mathbb{E}_\mu^{\pi_{\theta^{old}}}[TV(\pi_{\theta^{old}}(\cdot; S_t), \pi_\theta(\cdot, S_t))],$$
$$M_{ax}\, TV(\pi_{\theta^{old}}, \pi_\theta) := \max_{t=0,\dots,T-1} \mathbb{E}_\mu^{\pi_{\theta^{old}}}[TV(\pi_{\theta^{old}}(\cdot; S_t), \pi_\theta(\cdot, S_t))].$$

This yields

$$\|\nabla_\theta J(\theta) - g_{PPO}(\theta, \theta^{old})\| \leq const. \min\{M_{ean}\, TV(\pi_{\theta^{old}}, \pi_\theta), M_{ax}\, TV(\pi_{\theta^{old}}, \pi_\theta)\}.$$

To show this inquality one would first show that

$$\nabla_\theta J(\theta) = \nabla_\theta(V^{\pi_\theta}(\mu) - V^{\pi_{\theta^{old}}}(\mu)) = \nabla_\theta \sum_{t=0}^{T-1} \gamma^t \mathbb{E}_\mu^{\pi_\theta}[A_t^{\pi_{\theta^{old}}}(S_t, A_t)].$$

and that

$$\nabla_\theta J(\theta) - g_{PPO}(\theta, \theta^{old}) = \sum_{t=0}^{T-1} \gamma^t \nabla_\theta \left(\mathbb{E}^{\pi_\theta}\left[\mathbb{E}_{A\sim\pi_\theta(\cdot;S_t)}[A_t^{\pi_{\theta^{old}}}(S_t, A_t)]\right] - \mathbb{E}^{\pi_{\theta^{old}}}\left[\mathbb{E}_{A\sim\pi_\theta(\cdot;S_t)}[A_t^{\pi_{\theta^{old}}}(S_t, A_t)]\right]\right)$$

Remark 4.3

(**)

Theorem 4.1

Under the assumptions stated in the beginning

$$\exists R > 0: \quad |\nabla_\theta J(\theta) - g_{PPO}^{clip}(\theta, \theta^{old})| \leq R|\theta - \theta^{old}|.$$

This theorem shows that as long as the parameters remain in proximity (trust region) of θ^{old} the bias in the gradient stepp will also be bounded. Thus, g_{PPO}^{clip} with all its advantages (less correcaltion, mini batches of transitions and not entire rollouts and more value network updates (???)) is applicable.
The two expectations are approximated every cycle (Dataset containing transitions from π^{old})

Algorithm 2: Cyclic Reshuffled PPO Surrogate Ascent

Require: Initial θ, stepsize η, cycles C, epochs K, batch size B, $m := N/B$.

for $c = 0, \dots, C-1$ ▷ `cycles`
  do
    $\theta_{\text{old}} \leftarrow \theta$ ▷ `fixed sampling parameter`
    Sample n rollouts under $\pi_{\theta_{\text{old}}}$
    Estimate advantages $\hat{A}$ ▷ `critic step, e.g. using GAE`
    Fill buffer $\{(s^i, a^i, r^i, \hat{A}^i, t^i)\}_{i=0}^{N-1}$
    for $e = 0, \dots, K-1$ ▷ `epoch`
      do
        Draw a random permutation $\sigma = (\sigma_0, \dots, \sigma_{N-1})$
        for $k = 0, \dots, m-1$ ▷ `minibatch updates`
          do
            $\mathcal{B}_k \leftarrow \{\sigma_{kB}, \dots, \sigma_{(k+1)B-1}\}$
            Compute $\hat{g}^{\text{clip}}(\theta, \theta_{\text{old}}; \mathcal{B}_k) = \frac{1}{B} \sum_{i \in \mathcal{B}_k} T \gamma^{t_i} \frac{\nabla_\theta \pi_\theta(a^i, s^i)}{\pi_{\theta old}(a^i, s^i)} \mathbf{1}_{\frac{\pi_\theta(a^i, s^i)}{\pi_{\theta old}(a^i, s^i)}}$
            $\theta_{c,e,k+1} \leftarrow \theta_{c,e,k} + \eta\, \hat{g}^{\text{clip}}(\theta_{c,e,k}, \theta_{\text{old}}; \mathcal{B}_k)$
        end
    end
end

Here $\theta_{c,e,k}$ denote the parameter at the cycle, epoch and minibatch.

5 Advantage Estimation

Remark 5.1

GAE for infinite MDPs: Uses the k-step Bellman expectation operator. For large k we have Monte carlo and high variance. For small k we have high bias.

Given $\hat{V}$ the one step TD-error at time t is

$$\delta_t := R_t + \gamma \hat{V}(S_{t+1}) - \hat{V}(S_t).$$

If $\hat{V} = V^\pi$ then δ_t is unbiased:

$$\mathbb{E}[\delta_t \mid S_t, A_t] = \mathbb{E}[R_t + \gamma V^\pi(S_{t+1}) \mid S_t, A_t] - \mathbb{E}[V^\pi(S_t) \mid S_t, A_t] \overset{measurable}{=} Q^\pi(S_t, A_t) - V^\pi(S_t) = A^\pi(S_t, A_t).$$

The k-step advantage is

$$\hat{A}_t^{(k)} := \sum_{l=0}^{k} \gamma^l \delta_{t+l} = \sum_{l=0}^{k} \gamma^l R_{t+l} + \gamma^{k+1} \hat{V}(S_{t+k+1}) - \hat{V}(S_t).$$

If $\hat{V} = V^\pi$, then again

$$\mathbb{E}[\hat{A}_t^{(k)} \mid S_t, A_t] = A^\pi(S_t, A_t)$$

is unbiased.

Remark 5.2

The GAE (General advantage estimator) combines all k-step estimators using exponentially decaying weights controlled by $\lambda \in [0,1]$. Because the geometric distribution with $\mathbb{P}(X=k) = (1-\lambda)^{k-1}\lambda$ is the only memoryless distribution, it is compatible with the Markov property. $TD(\lambda)$ schemes interpolate between one step updates and Monte Carlo. The forward estimator extends the n-step advantage, which is not very efficient. The backward scheme uses past values for the update.

$$\hat{A}_t^{GAE(\lambda)} := (1-\lambda)\sum_{k=0}^{\infty} \lambda^k \hat{A}_t^{(k)}.$$

Using the telescoping sum, it can be rewritten as

$$\hat{A}_t^{GAE(\lambda)} = \sum_{l\in\mathbb{N}} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = R_t + \gamma V(S_{t+1}) - V(S_t).$$

Equivalently it can be recursively defined

$$\hat{A}_t^{GAE(\lambda)} := \delta_t + \gamma\lambda \hat{A}_{t+1}^{GAE(\lambda)}.$$

For $\lambda = 0$ it is one step TD-error δ_t. For $\delta \to 1$ it is the Monte Carlo estimator minus the current value function.
Note that this estimator assume access to all k-step estimators, i.e. access to infinite trajectories. In practice, trajectories are never infinite. So it is reasonable to only consider a finite time horizon. However, truncating the estimator destroys the geometric weighing.

Remark 5.3

Fixed-Time Advantage estimation: Here the infinite exponential weighing has to be corrected for the finite time horizon. $\forall t < T < \infty$ define

$$\hat{A}_t^T := \frac{1-\lambda}{1-\lambda^{T-t}} \sum_{k=0}^{T-t-1} \lambda^k \hat{A}_t^{(k)},$$

where $\hat{A}_t^{(k)}$ is again the k-step advantage. The normalization factor $\frac{1-\lambda}{1-\lambda^{T-t}}$ ensures that the weights sum to one. This is the finite horizon analogue to GAE.

Heuristically, the forward estimator is least affected by bootstrapping, but it has higher variance, as it aggregates large discounted sums of TD errors. In general it is smart to choose λ close to 1 in the beginning, as then the advantages A_t^k with larger k (longer rollouts) are weighted stronger. This is because in the beginning the l^2 error of V^π and its approximation is higher. As time progresses, one can then reduce λ, as

the error of approximation decreases. This can also be seen in the following:

$$\mathbb{E}^{\pi}_{s,a}\left[\left(\sum_{t=0}^{n-1}\gamma^t R_t + \gamma^n Q(S_, A_n) - Q^{\pi}(s,a)\right)^2\right]$$
$$\leq \underbrace{\gamma^{2n}\mathbb{E}^{\pi}_{s,a}\left[Q^{\pi}(S_n,A_n) - Q(S_n,A_n)\right]^2}_{\text{old estimation bias}} + \underbrace{\mathbb{V}^{\pi}_{s,a}\left[\sum_{t=0}^{n-1}\gamma^t R_t\right]}_{\text{Monte Carlo Variance}} + \underbrace{\gamma^{2n}\mathbb{V}^{\pi}_{s,a}[Q(S_n,A_n)]}_{\text{old estimation variance}}.$$

If the old estimation bias is large (in the beginning) choose n large (because γ^{2n} will be small).

Lemma 5.1: Backward Recursion fixed time

For $\hat{A}^T_T = 0$, it holds $\forall t < T$:

$$\hat{A}^T_t = \delta_t + \gamma\lambda\frac{1-\lambda^{T-t-1}}{1-\lambda^{T-t}}\hat{A}^T_{t+1}.$$

Note that $\frac{1-\lambda^{T-t-1}}{1-\lambda^{T-t}}$ approaches one as $t \to T$.

Remark 5.4

Often trajectories end when reaching a terminating state. In such a cases it no longer makes sense to consider k-step averages beyond that point. So our estimator depends on this stopping time. Define the natural filtration $(\mathcal{F}_t)_{t\geq 0}$ by

$$\mathcal{F}_t := \sigma(S_0, A_0, R_0, ..., S_t, A_t),$$

all information up to time t. Define

$$\tilde{\tau} := \inf\{t \geq 0 \mid S_t \text{ is terminal }\}$$

which is a stopping time wrt. $(\mathcal{F}_t)_{t\geq 0}$. Assume that $\tilde{\tau}$ is finite, then for fixed $T > 0$ define $\tau := \tilde{\tau} \wedge T$. Define

$$\hat{A}^{\tau}_t := \frac{1-\lambda}{1-\lambda^{\tau-t}}\sum_{k=0}^{\tau-t-1}\lambda^k\hat{A}^{(k)}_t.$$

If $\tau = T$, then we have the finite-horizon version $\hat{A}^T_t$.

Lemma 5.2

Let $\hat{A}^{\tau}_{\tau} = 0$, then

$$\hat{A}^{\tau}_t = \delta_t + \gamma\lambda\frac{1-\lambda^{\tau-t-1}}{1-\lambda^{\tau-t}}\hat{A}^{\tau}_{t+1}.$$

6 Variants to PPO

7 Stable Baselines3

- ReplayBuffer: Off Policy, it fills the Buffer wrt. some behavioral policy and then samples uniformly, the mini batches with whom the gradient steps are done. After the epoch is finished, the samples are put back. If the Buffer reaches its memory limitations, samples are discarded according to FIFO.
- RolloutBuffer: On Policy, it stores entire trajectories of the current policy. However, after an epoch it completely resets. It computes the GAE for fixed time via the backward recursion.
- NStepReplayBuffer: It follows some behavioral policy (off policy) to estimate n step advantages.

8 Hyperparameter Tuning

Remark 8.1

CMA ES:
To minimize $f : \mathbb{R}^d \to \mathbb{R}$ we iteratively update a sequence of $X^n \sim \mathcal{N}(m_n, \sigma_n^2 C_n)$ from which in each step N samples are drawn, i.e. $\forall n \in \mathbb{N}$ sample $X_1^{(n)}, ..., X_N^{(n)} \sim \mathcal{N}(m_n, \sigma_n^2 C_n)$ and order them such that

$$f(X_{1:N}^{(n)} \leq \cdots \leq f(X_{N:N}^{(n)})).$$

From the $K < N$ best candidates choose the next parameters as

$$m^{n+1} := \sum_{i=0}^{K} w_i X_{N-i:N}^{(n)}, \quad \text{where } w_1 > \cdots > w_K.$$

Remark 8.2

Pruning Trials: